

T3: MD-Confirm — Publish-Time Provenance Agent

Заявка на All Things Agentic Hackathon (трек Taskmaster)

Дедлайн: 31 августа 2026, 17:00 PDT **Автор концепции:** Юрий Иванов / Miracle Droplet MD™ **Статус документа:** рабочее T3 для сборки за ~12 дней

1. Резюме (elevator pitch)

Google Pixel уже подписывает каждое фото аппаратным C2PA-сертификатом в момент съёмки. Проблема в том, что **соцсети стирают эти метаданные при загрузке** — подтверждение подлинности умирает ровно в тот момент, когда контент становится публичным и начинает распространяться. Мы не конкурируем с C2PA/SynthID — мы строим агента, который **переживает публикацию**: в момент "поделиться" фиксирует хеш происхождения во внешнем неизменяемом реестре, привязывает его к конкретному посту на конкретной платформе, и позволяет любому проверить позже — уже после того, как исходные метаданные стёрты — было ли это фото верифицировано при съёмке.

Одна фраза для судей: "Google доказывает подлинность в момент съёмки. Мы доказываем, что это доказательство не потерялось, когда фото ушло в мир."

2. Проблема, которую решает агент

C2PA/Content Credentials — сильный стандарт, но данные о происхождении живут **внутри файла**. При загрузке в Instagram/TikTok/Facebook/X платформы транскодируют и стирают embedded-метаданные.

После этого у зрителя нет способа узнать, что фото было верифицировано при съёмке — только у автора, если он сам сохранил оригинал.

Это признанная, ещё не решённая индустрией проблема (см. отчёт Microsoft, февраль 2026: ни один метод в одиночку не решает задачу digital deception).

Ручное решение (автор сохраняет пруфы, показывает по запросу) — это именно та "messy multi-step chore", которую должен автоматизировать агент.

3. Трек хакатона и обоснование

Taskmaster — "Build a complete workflow, not just a chatbot... Build an agent that handles the details, sends the right info to the right places, and proves it can do the heavy lifting for you."

Наш workflow:

Пользователь снимает/выбирает фото → жмёт "Поделиться"

Агент **сам** извлекает C2PA-манифест, генерирует хеш-квитанцию, публикует её в реестр, дожидается подтверждения транзакции.

Агент **сам** решает: контент верифицирован полностью / частично / не верифицирован — и соответственно либо продолжает публикацию с "receipt ID", либо предупреждает пользователя.

После публикации — отдельный сервис агента отвечает на запросы верификации по URL поста, без участия пользователя.

Ни один шаг не требует ручного вмешательства после нажатия "Поделиться" — это autonomous multi-step action, а не чат.

4. Что НЕ делаем в рамках хакатона (осознанно вне scope)

Не делаем	Почему	Чем заменяем для демо
Реальный RAW/PRNU-захват сенсора	Требует глубокой интеграции с Camera2 API, недели работы	Симулируем: заранее подготовленный набор "verified" / "tampered" тестовых фото с синтетическими хешами
Продакшн Solana mainnet	Лишние расходы, риски инфраструктуры	Solana devnet — бесплатно, полностью рабочий блокчейн-уровень, честно показываем в демо
Реальную интеграцию с API Instagram/TikTok	Требует partner-доступа, недели апрувов	Свой mock "share endpoint", имитирующий поведение платформы (включая "стирание" метаданных) — показывает механику честно
Обучение ML-модели детекции дипфейков	Не нужно — детекция уже есть в C2PA/существующих SDK	Используем готовый python пакет для чтения C2PA-манифеста (c2pa-python)

Отдельно проговорить в видео: это демо-архитектура, доказывающая механику агента, не production-продукт.

5. Пользовательский сценарий (end-to-end)



[Фото с C2PA-манифестом]



[Пользователь: "Поделиться" → выбирает наш Share-обработчик]



ON-DEVICE (Android, лёгкий слой)	
- извлекает C2PA-манифест из файла	
- если манифеста нет → флаг "unverified"	
- отправляет ТОЛЬКО метаданные	
(hash, timestamp, device attestation)	
в облако — САМО ФОТО не уходит	



CLOUD AGENT (Gemini + ADK, Cloud Run)	
1. Верифицирует C2PA-подпись	
2. Решает: verified / partial / none	
3. Регистрирует receipt в реестре	
(Firestore + Solana devnet anchor)	
4. Возвращает receipt_id + verdict	
5. Если "none" при попытке выдать	
фото за проверенное → эскалация:	
предупреждение пользователю,	
запись в audit-лог	



[Публикация продолжается + receipt_id прикреплён
как ссылка в описании/alt-тексте поста]



POST-PUBLICATION LOOKUP (тот же агент)	
Любой человек переходит по receipt_id →	
агент отвечает: "Да, этот контент был	
верифицирован при съёмке [timestamp],	
устройство [attested], хеш совпадает"	

6. Архитектура системы

6.1 On-device слой (минимальный, для демо — эмулятор/тестовое Android-приложение)

Простой Android-компонент (Kotlin), который:

читает C2PA-манифест из JPEG (библиотека c2pa-rs / c2pa-python на бэкенде проще — можно парсинг делать на облаке)

формирует payload: `{image_hash, c2pa_signature_valid: bool, capture_timestamp, device_attestation_id}`

отправляет payload в Cloud Agent через REST/gRPC

Важно: сырое фото не покидает устройство в облако агента — только метаданные (сохраняем privacy-принцип из MD-Confirm/MD-Spectrum)

6.2 Cloud Agent (ядро хакатон-проекта)

Gemini 3.5 (через Vertex AI) — слой рассуждения и принятия решения

Google ADK — оркестрация мультишагового workflow (verify → decide → register → respond)

Cloud Run — хостинг агента (обязательное требование хакатона)

Firestore — state/memory: receipts, история решений по устройствам/аккаунтам, audit-лог

Pub/Sub (опционально, если успеваем) — асинхронная обработка регистрации в блокчейне, чтобы не блокировать UX

6.3 Blockchain anchor

Solana devnet: транзакция с мемо = `image_hash + timestamp + agent_decision`

Даёт неизменяемую метку времени независимо от Google/платформы — это отличает решение от "просто Firestore"

7. Технологический стек (маппинг на обязательные требования хакатона)

Требование хакатона	Что используем
Gemini 3.5+ через API/Vertex AI	Vertex AI Gemini 3.5 Flash — reasoning-слой принятия решений
Google Agent Framework	Google ADK — оркестрация шагов verify→decide→register→respond, управление state
Google Cloud infra	Cloud Run (хостинг агента) + Firestore (state/memory)
(доп., не обязательно)	Pub/Sub для асинхронной публикации в блокчейн

Дополнительно:

Solana devnet (web3.js/solana-py) — внешний anchor

c2pa-python — чтение/валидация C2PA-манифестов

FastAPI — обёртка REST-эндпоинтов вокруг ADK-агента

8. Спецификация поведения агента

8.1 Состояния (states)

`RECEIVED` — метаданные получены

`C2PA_VALID` / `C2PA_MISSING` / `C2PA_INVALID` — результат проверки подписи

`REGISTERED` — receipt зафиксирован в Firestore + отправлен в очередь на блокчейн-anchor

`ANCHORED` — подтверждена транзакция в Solana devnet

`FLAGGED` — попытка выдать неverified контент за verified (эскалация)

8.2 Правила принятия решений (few-shot, не ML-обучение)

Зашиваются в системный промпт агента как сценарии, например:

Манифест валиден + подпись устройства совпадает с известным attested-устройством → `verified`, регистрируем без вопросов

Манифеста нет вообще, но пользователь не заявляет "это подлинное фото" (обычный шэр) → `unverified`, просто помечаем, не блокируем

Манифеста нет, но метаданные поста/подпись пользователя утверждают "оригинал/несмонтировано" →

`FLAGGED`, агент **сам** решает придерживаться регистрацию и уведомить пользователя с объяснением почему

Манифест есть, но hash не совпадает с фактическим файлом (подмена после подписи) → `FLAGGED`,

`tampered`, лог инцидента

Это НЕ тренировка модели — это набор из 8–10 явных правил-примеров в промпте ADK-агента, на которые Gemini опирается при рассуждении.

8.3 State & memory

Firestore коллекция `receipts`: image_hash, decision, timestamp, device_id (attested, не PII), solana_tx_id

Firestore коллекция `device_history`: сколько раз устройство присылало valid/invalid — нужно для контекстных решений ("это устройство и раньше присылало флагованный контент")

Agent Runtime (ADK) держит короткую сессионную память на время одного workflow (verify→register), долгую память — только через Firestore

8.4 Секьюрность/credentials

Solana wallet private key — в Google Secret Manager, не в коде и не в переменных окружения репозитория Vertex AI/Cloud Run — Service Account с минимальными правами (принцип наименьших привилегий)

On-device → Cloud: подписанные запросы (HMAC) с ключом, привязанным к attested-устройству

8.5 Обработка сбоев (failure handling)

Gemini API недоступен → агент переходит в детерминированный fallback-режим (просто регистрирует hash без reasoning-слоя, помечает `degraded_mode: true`)

Solana devnet недоступен → receipt остаётся в статусе `REGISTERED` (Firestore), retry через Pub/Sub с экспоненциальным backoff, anchor подтягивается асинхронно

Показать в демо намеренный сбой одного из сервисов и то, как агент деградирует gracefully — это прямое попадание в критерий "handle failures"

9. Пример данных (Firestore schema)

☐
json

```
// collection: receipts
{
  "receipt_id": "rcpt_8f3a...",
  "image_hash": "sha256:...",
  "c2pa_status": "valid",
  "device_attestation_id": "attest_xxxx",
  "decision": "verified",
  "capture_timestamp": "2026-08-20T10:15:00Z",
  "registered_at": "2026-08-20T10:15:03Z",
  "solana_tx_id": "5h3k...",
  "solana_status": "anchored"
}
```

10. API-эндпоинты агента

Метод	Путь	Назначение
POST	<code>/verify</code>	Принимает метаданные с устройства, запускает ADK workflow
GET	<code>/receipt/{receipt_id}</code>	Публичный lookup — "было ли это верифицировано"
GET	<code>/device/{device_id}/history</code>	Внутренний — история решений по устройству (для reasoning)
POST	<code>/webhook/solana-confirm</code>	Callback подтверждения транзакции

11. Демо-сценарий (~4 минуты видео)

0:00-0:30 — Проблема: показать реальный пример, как соцсеть стирает C2PA-метаданные (до/после скачивания с Instagram — метаданные исчезли)

0:30-1:00 — "Вот наш агент" — коротко архитектура (диаграмма на экране)

1:00-2:30 — Живое демо:

берём фото с валидным C2PA → жмём "поделиться через MD-Confirm Agent"

показываем реальный лог в Cloud Run/Vertex AI console — видно, как Gemini принимает решение

показываем запись в Firestore и подтверждённую транзакцию в Solana devnet explorer

открываем receipt-ссылку — "verified  снято 20.08.2026, устройство подтверждено"

2:30-3:15 — Сценарий FLAGGED: пытаемся протащить фото без манифеста с подписью "оригинал" → агент сам ловит и блокирует/предупреждает, объясняем почему это multi-step autonomous decision, а не if/else скрипт

3:15-3:45 — Намеренно роняем Gemini API → показываем graceful degradation

3:45-4:00 — Итог + ссылка на репозиторий

12. Чеклист сдачи (по требованиям Devpost)

- ☐ Категория: Taskmaster
- ☐ URL хостед-проекта (если успеваем задеплоить публично)
- ☐ Текстовое описание: features, technologies, data sources, findings/learnings

- ☐ Публичный/приватный репозиторий + README со Spin-up Instructions (доступ для testing@devpost.com и clouhackathons@google.com, если приватный)
- ☐ Архитектурная диаграмма (визуальная версия схемы из раздела 5)
- ☐ Демо-видео ~4 мин (см. сценарий выше), с явным доказательством работы на Google Cloud (Cloud Run dashboard/Vertex AI logs)
- ☐ (бонус) Публикация о том, как строили проект + пост в соцсетях с #AllThingsAgenticHackathon

13. План на ~12 дней

Дни	Задача
1-2	Настройка GCP-проекта, Vertex AI доступ, Cloud Run скелет, ADK hello-world агент
3-4	Интеграция c2pa-python (чтение манифеста), Firestore schema, базовый <code>/verify</code> endpoint
5-6	Логика правил принятия решений в ADK (few-shot промпт), состояния верификации
7-8	Solana devnet интеграция (anchor), Pub/Sub для асинхронности, <code>/receipt/{id}</code> lookup
9	Обработка сбоев/деградации, Secret Manager для ключей
10	Мини-Android/веб клиент для демо share-flow (можно просто веб-форма вместо полного Android-приложения)
11	Съёмка демо-видео, архитектурная диаграмма, README
12	Буфер на баги, финальная сборка сабмишена, публикация

14. Соответствие критериям судейства

Критерий	Вес	Как закрываем
Innovation & Operational Utility	40%	Решаем реальную, признанную индустрией нерешённой проблему (стирание провенанса при публикации), агент принимает решения (verified/flagged/degraded) без хардкода if/else, автономно на каждом шаге
Architectural Discipline & Tech Stack	30%	Чёткое разделение on-device/cloud, state в Firestore, секреты в Secret Manager, graceful degradation при сбое Gemini/Solana, асинхронность через Pub/Sub
Demo & Production Readiness	30%	Живое демо с видимыми Cloud Run/Vertex AI логами, воспроизводимый README, честная архитектурная диаграмма

15. Риски и как их обходим

Риск	Митигация
Не успеваем реальную Android-интеграцию	Заменяем на веб-демо (upload формы), сама механика агента важнее клиента
Solana devnet нестабилен во время демо	Заранее делаем 2-3 успешных прогона, записываем как fallback-клип на случай сбоя во время живой съёмки
Жюри знает про C2PA/Pixel и посчитает вторичным	Чётко проговариваем в видео с первых 30 секунд: "мы не переизобретаем C2PA, мы решаем то, что C2PA признанно не решает" — со ссылкой на отчёт Microsoft
Слишком амбициозный score за 12 дней	Раздел 4 ("что не делаем") — держать эту границу жёстко весь спринт

16. Стратегическое видение (упомануть в видео/описании, не строить)

Anchor-слой на хакатоне реализован на **Solana devnet** — открытый, бесплатный, доступен любому разработчику без онбординга.

Google параллельно строит собственный Layer-1 — **Google Cloud Universal Ledger (GCUL)**: permissioned блокчейн для платежей и токенизации активов, сейчас в закрытом тестнете (пилот с CME Group), с KYC-онбордингом для институтов, публичный коммерческий запуск ожидается в 2026 году. Доступа для сторонних разработчиков/хакатон-участников сейчас нет — поэтому не используем его в сборке.

Стоит явно проговорить в тексте заявки (не в коде): как только GCUL откроет доступ для верифицированных организаций, anchor-слой можно перенести туда — это даст более "нативную" для Google-экосистемы историю (единый провайдер: Vertex AI + Cloud Run + GCUL), не завязанную на внешний блокчейн. Это не меняет архитектуру демо, но показывает судьям стратегическое понимание экосистемы, в которой они сами строят продукт.